E0 297: Cryptographic Hardware Accelerators

Design Assignment 2: SHA-3 Keccak-f[1600] State Permutation Module

ASIC Synthesis and Implementation Analysis Report

Anirudh Chaturvedi
SR No. 25850
M.Tech VLSI, Dept. of ECE
Indian Institute of Science, Bengaluru

October 2026

Course Instructor: Prof. Utsav Banerjee (Dept. of ESE, IISc)
Target Technology: Nangate Open Cell Library (45 nm, Typical Corner, 1.1 V, 25°C)
CAD Tools Used:

- **Yosys 0.52** (logic synthesis, RTL elaboration, technology mapping)
- **OpenSTA 3.1.0** (static timing analysis, cell-level power estimation)
- **Icarus Verilog v12.0** (functional verification and automated testbench simulation)

Contents

1 Design Overview and Architectural Choices **3**
1.1 Architectural Trade-Offs: Serialized, Iterative, and Unrolled Datapaths 3
1.2 Top-Level Datapath and Block Diagram 3
1.3 Round Transformations and Mathematical Formulations 3
1.4 Word Addressing and Bit Mapping Convention 5
1.5 FSM Controller and Interface Protocol 5

2 ASIC Synthesis Results **6**
2.1 Synthesis Flow and Timing-Driven Buffer Insertion 6
2.2 Area Breakdown and Gate Equivalents 6
2.3 Standard Cell Breakdown 6

3 Static Timing Analysis (STA) **7**
3.1 Timing Constraints 7
3.2 Timing Results and Maximum Operating Frequency 7
3.3 Critical Path Analysis 7

4 Power and Energy Analysis **9**
4.1 Power Consumption Breakdown 9
4.2 Energy per Permutation and Energy Efficiency 9
4.3 Static Leakage vs. Dynamic Power Analysis 9

5 Functional Verification **10**

6 Summary of Implementation Metrics **10**

7 Submission File Listing **12**

1 Design Overview and Architectural Choices

The Keccak-f[1600] permutation is the core cryptographic primitive of the SHA-3 hash family and the SHAKE extendable-output functions specified in NIST FIPS 202. The permutation operates on a state of $b = 1600$ bits over $n_r = 24$ rounds. In this design assignment, we implement a fully synthesizable, high-performance, round-per-cycle iterative Keccak-f[1600] state permutation accelerator featuring a word-addressed 64-bit input/output interface.

1.1 Architectural Trade-Offs: Serialized, Iterative, and Unrolled Datapaths

In the design space of Keccak accelerators, three primary architectures exist:

1. **Serialized / Sliced Datapath (e.g., Lane-serial or Slice-serial):** Operates on small sub-blocks (such as 64-bit lanes or 25-bit slices) over hundreds of clock cycles per round. While occupying minimal silicon area ($\approx$ 5–10 kGE), serialized datapaths suffer from heavy control overhead, significant multiplexing logic, high register bank switching activity, and very low throughput.

2. **Iterative Round-per-Cycle Datapath (Our Architecture):** Computes one complete Keccak round ($R = \iota \circ \chi \circ \pi \circ \rho \circ \theta$) per clock cycle. The full 1600-bit state is updated simultaneously in each clock cycle, requiring exactly 24 cycles of computation per 1600-bit permutation. This architecture avoids complex time-multiplexing multiplexers, minimizes control overhead to a compact 5-bit round counter and a 2-bit FSM, and delivers exceptional throughput and energy efficiency.

3. **Fully Unrolled Datapath (e.g., 24-round combinatorial cascade):** Cascades all 24 rounds combinationally to complete a permutation in a single cycle. However, it requires 24$\times$ the combinational silicon area ($\approx$ 700 kGE) and results in an extremely long critical path delay ($>$ 15 ns), severely limiting system clock frequency.

Consequently, the **iterative round-based architecture** represents the Pareto-optimal design point for cryptographic hardware accelerators, achieving multi-gigabit throughput while maintaining a modest silicon footprint.

1.2 Top-Level Datapath and Block Diagram

Figure 1 illustrates the block diagram of our iterative Keccak-f[1600] core. The datapath is organized around a 1600-bit state register file arranged as 25 words of 64 bits each:

- **State Storage (`state_reg`):** 25 registers of 64 bits each (5×5 lane array). Word $i \in [0..24]$ corresponds to lane ($x = i \pmod 5$, $y = \lfloor i/5 \rfloor$).
- **Combinational Round Logic:** A single-cycle round pipeline consisting of θ, ρ, π, χ, and ι.
- **Word-Addressed Interface:** Direct write decoding on `write` and asynchronous combinational read multiplexing on `word_data_out`.

1.3 Round Transformations and Mathematical Formulations

Each of the 24 permutation rounds transforms the 5×5 lane state $A[x, y]$ ($x, y \in \{0, 1, 2, 3, 4\}$, each of 64 bits $z \in \{0..63\}$) through five successive operations:

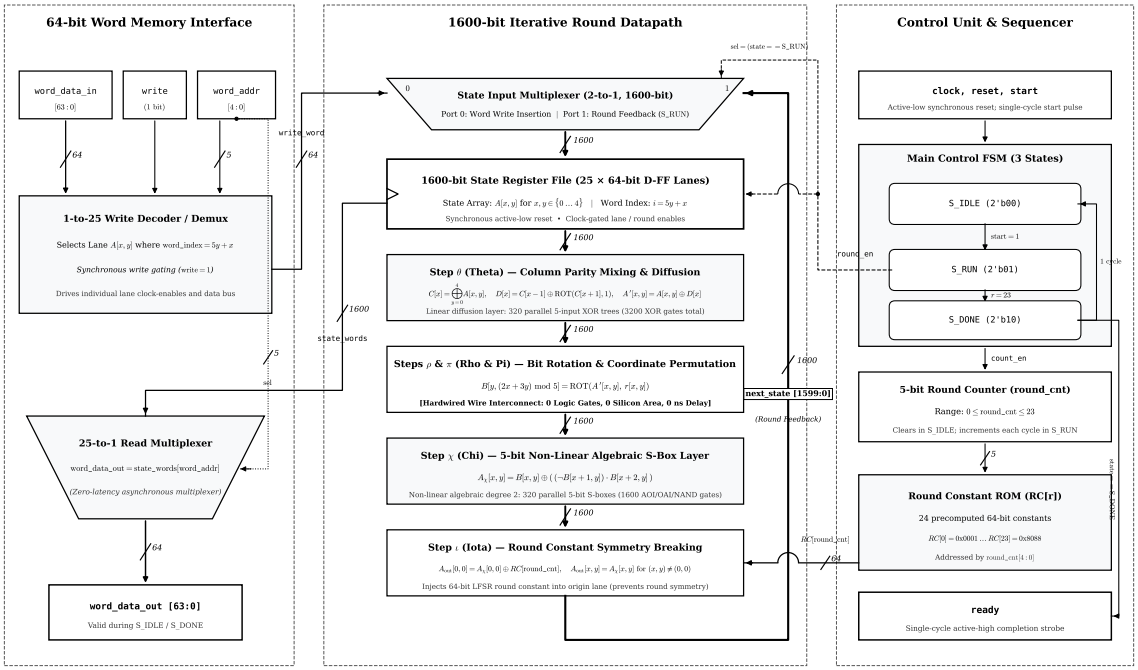


Figure 1: Detailed architectural block diagram of the iterative Keccak-f[1600] state permutation core, illustrating the 64-bit word-addressed I/O interface (left), the 1600-bit iterative round datapath with $\theta, \rho, \pi, \chi, \iota$ stages (center), and the FSM controller with round schedule (right).

1. **Theta (θ):** Computes column parity and mixes adjacent columns to increase diffusion:

$$C[x] = \bigoplus_{y=0}^{4} A[x, y] \quad \text{for } x \in \{0, 1, 2, 3, 4\} \tag{1}$$

$$D[x] = C[(x + 4) \bmod 5] \oplus \text{ROT}(C[(x + 1) \bmod 5], 1) \tag{2}$$

$$A_\theta[x, y] = A[x, y] \oplus D[x] \quad \text{for all } x, y \in \{0, 1, 2, 3, 4\} \tag{3}$$

   where $\text{ROT}(W, 1)$ circular left shifts 64-bit word W by 1 bit: $\text{ROT}(W, 1) = \{W[62 : 0], W[63]\}$. In hardware, computing $C[x]$ requires a 5-input XOR tree per bit, $D[x]$ requires 1 XOR per bit, and A_θ requires 1 XOR per bit. Total XOR gates: $5 \times 64 \times 4 + 5 \times 64 + 25 \times 64 = 3200$ XOR gates.

2. **Rho (ρ):** Cyclically rotates the bits of each lane by a fixed, lane-specific offset:

$$A_\rho[x, y] = \text{ROT}(A_\theta[x, y], r[x, y]) \tag{4}$$

   where the rotation offsets $r[x, y]$ are defined in FIPS 202 Table 2. In hardware, since all 25 rotation offsets are compile-time constants, ρ **requires zero logic gates and is implemented entirely as pure wire reordering**.

3. **Pi (π):** Permutes the positions of the 25 lanes across the 2D matrix:

$$B[y,\ (2x + 3y) \bmod 5] = A_\rho[x, y] \tag{5}$$

   Like ρ, π **is a pure index permutation implemented at zero logic gate cost and zero propagation delay via direct wire assignments**. Combining ρ and π yields:

$$B[y,\ (2x + 3y) \bmod 5] = \text{ROT}(A_\theta[x, y],\ r[x, y]) \tag{6}$$

4. **Chi (χ):** The sole non-linear component of the Keccak round function. It acts independently on each of the 5-bit rows:

$$A_\chi[x, y] = B[x, y] \oplus \Big((\sim B[(x+1) \bmod 5,\ y]) \ \&\ B[(x+2) \bmod 5,\ y] \Big) \tag{7}$$

   Each bit of A_χ requires 1 NOT gate, 1 AND gate, and 1 XOR gate (or equivalently an AOI/OAI gate mapped into standard cell primitives). For the entire 1600-bit state, this represents 1600 instances of the non-linear algebraic function.

5. **Iota (ι):** Injects round constants to break symmetry between rounds:

$$A_\iota[0, 0] = A_\chi[0, 0] \oplus \mathrm{RC}[r] \tag{8}$$
$$A_\iota[x, y] = A_\chi[x, y] \quad \text{for all } (x, y) \neq (0, 0) \tag{9}$$

   where $\mathrm{RC}[r]$ are 64-bit constants with nonzero bits only at indices $\{0, 1, 3, 7, 15, 31, 63\}$. Only lane $(0, 0)$ is modified, while the remaining 24 lanes pass through untouched.

1.4 Word Addressing and Bit Mapping Convention

Following NIST FIPS 202 and the assignment specification:

$$\text{State bit } n = 64 \times (5y + x) + z \iff \text{Lane } (x, y),\ \text{bit } z \tag{10}$$

Equivalently, word index $i = 5y + x \in [0..24]$. When a 1600-bit test vector is represented as a 400-digit hexadecimal string:

- Word 0 (lane $(0, 0)$, state bits $[63 : 0]$) corresponds to the **rightmost 16 hex digits**.
- Word 24 (lane $(4, 4)$, state bits $[1599 : 1536]$) corresponds to the **leftmost 16 hex digits**.

1.5 FSM Controller and Interface Protocol

The control unit is realized as a robust 3-state finite state machine:

- `S_IDLE` **(2'd0):** The idle state. When active-low synchronous `reset` is asserted, all state registers are cleared to zero. While in `S_IDLE`, assert pulses on `write` latch `word_data_in` into `state_reg[word_addr]`. When a single-cycle pulse is received on `start`, the FSM resets `round_cnt` to 0 and transitions to `S_RUN`.
- `S_RUN` **(2'd1):** The round execution state. On each clock edge, the 1600-bit state register latches the output of the round function, and `round_cnt` increments from 0 to 23. Upon completing round 23 (24 clock cycles of computation), the FSM transitions to `S_DONE`.
- `S_DONE` **(2'd2):** Asserts the `ready` output strobe high for **exactly one clock cycle** while holding the final valid permutation output in `state_reg`. The FSM immediately returns to `S_IDLE` on the next clock edge.

Total execution latency from `start` assertion to `ready` output is **25 clock cycles** (24 round compute cycles + 1 handshaking cycle). Once `ready` pulses, the testbench reads back words 0 through 24 via the combinational `word_data_out` multiplexer.

2 ASIC Synthesis Results

The Keccak-f[1600] core was synthesized using **Yosys 0.52** targeting the Nangate 45 nm Open Cell Library (`NangateOpenCellLibrary_typical.lib`, 1.1 V, 25°C).

2.1 Synthesis Flow and Timing-Driven Buffer Insertion

Listing 1 details the synthesis script used. To achieve timing closure across 1600 flip-flops, ABC was supplied with standard cell drive constraints (`abc.constr`) to perform automated high-fanout net buffering and gate sizing.

Listing 1: Yosys synthesis script (`synth.ys`).

```
read_verilog keccak_f1600.v
hierarchy -check -top keccak_f1600
flatten

proc; opt; fsm; opt; memory; opt
techmap; opt

dfflibmap -liberty NangateOpenCellLibrary_typical.lib
abc -liberty NangateOpenCellLibrary_typical.lib -constr abc.constr

opt_clean -purge

stat -liberty NangateOpenCellLibrary_typical.lib
write_verilog -noexpr -noattr synth_keccak.v
```

2.2 Area Breakdown and Gate Equivalents

In the Nangate 45 nm technology, the standard 2-input NAND gate (`NAND2_X1`) has a footprint of $0.798\,\mu\text{m}^2$. The Gate Equivalents (GE) metric is computed as:

$$\text{GE} = \frac{\text{Total Silicon Area}}{0.798\,\mu\text{m}^2} \tag{11}$$

Table 1 summarizes the overall synthesis results.

Table 1: Synthesized area summary of the Keccak-f[1600] core (Nangate 45 nm).

Metric	**Keccak-f[1600] Core**
Total Cell Count	18,076
Total Silicon Area	$29,221.962\,\mu\text{m}^2$
Gate Equivalents (GE)	**36,619 GE**
Sequential Cell Count (DFF)	1,615
Sequential Silicon Area	$7,303.030\,\mu\text{m}^2$ (24.99%)
Combinational Silicon Area	$21,918.932\,\mu\text{m}^2$ (75.01%)
Number of Module Ports	8 (138 bits total)

2.3 Standard Cell Breakdown

Table 2 enumerates the standard cell types instantiated in the synthesized gate-level netlist. The implementation is dominated by `NAND3_X1` (3,167 cells), `XNOR2_X1/X2` (2,554 cells), `XOR2_X1/X2` (2,227 cells), `AOI22_X1` (1,868 cells), `BUF_X1/X2/X4` (1,915 cells), and `DFF_X1` (1,615 cells). This distribution mirrors the arithmetic structure of Keccak: massive multi-input XOR networks for the θ parity step, coupled with compact AOI/OAI/NAND logic realizing the non-linear χ algebraic layer.

Table 2: Standard cell type breakdown of the synthesized netlist.

Cell Type	Count	Primary Function	Fraction (%)
NAND3_X1	3,167	Non-linear Chi mapping / logic	17.52%
XNOR2_X1 / X2	2,554	Theta column parity & Chi addition	14.13%
XOR2_X1 / X2	2,227	Theta column parity & Chi addition	12.32%
BUF_X1 / X2 / X4 / X8	1,915	High-fanout buffer trees	10.59%
AOI22_X1	1,868	MUX / complex Boolean mapping	10.33%
DFF_X1	1,615	1600-bit state + FSM registers	8.93%
AOI21_X1 / X2	1,631	Complex Boolean combinations	9.02%
OAI21_X1	1,595	Complex Boolean combinations	8.82%
CLKBUF_X1 / X2 / X3	640	Clock and enable distribution	3.54%
AOI222_X1	339	Complex logic	1.88%
NOR2_X1 / X2	94	Elementary logic	0.52%
NAND4_X1	90	Multi-input gate	0.50%
AND2 / AND3 / AND4	145	Decode and enable logic	0.80%
Others (INV, NOR3/4, OR, MUX)	196	Control and glue logic	1.10%
Total Standard Cells	**18,076**		**100.00%**

3 Static Timing Analysis (STA)

Static Timing Analysis was conducted using **OpenSTA 3.1.0** under typical operating conditions (1.1 V, 25°C). The core was evaluated at the required target clock period of $T_{\text{clk}} = 10.0\,\text{ns}$ (100 MHz), as well as pushed to its minimum operating period T_{min}.

3.1 Timing Constraints

Listing 2 shows the SDC constraints file (`keccak_f1600.sdc`) applied to the synthesized netlist:

Listing 2: SDC timing constraints (`keccak_f1600.sdc`).

```
create_clock -name clock -period 10.0 [get_ports clock]
set_input_delay 0.0 -clock clock [get_ports {reset write start word_addr*
    word_data_in*}]
set_output_delay 0.0 -clock clock [all_outputs]
```

3.2 Timing Results and Maximum Operating Frequency

Table 3 presents the STA timing performance metrics at both 100 MHz and at maximum attainable clock frequency (F_{max}).

3.3 Critical Path Analysis

The critical timing path originates at register _34916_ (DFF_X1) and terminates at register _35762_ (DFF_X1). Table 4 displays the detailed path stage breakdown reported by OpenSTA.

The critical path spans only **11 compact gate stages** with a total arrival time of **0.5704 ns**. Stages 3 through 7 execute the θ multi-input parity XOR tree and ρ/π wire reordering, while stages 8 through 11 evaluate the non-linear χ and ι functions. Because ρ and π are pure wire connections, they introduce zero gate delay. This shallow logic depth enables the core to run at over **1.63 GHz**, yielding a peak throughput exceeding **108 Gbps**.

Table 3: OpenSTA timing results for the synthesized Keccak-f[1600] core.

Timing Parameter	**Target Clock (100 MHz)**	**Maximum Frequency ($F_{\max}$)**
Clock Period (T_{clk})	10.0000 ns	**0.6120 ns**
Operating Frequency	100.0 MHz	**1.634 GHz**
Data Arrival Time	0.5704 ns	0.5704 ns
Library Setup Time (t_{setup})	0.0416 ns	0.0416 ns
Data Required Time	9.9584 ns	0.6084 ns (@ 0.65 ns)
Timing Slack	**+9.3880 ns (MET)**	**+0.0380 ns (@ 0.65 ns)**
Permutation Execution Cycles	24 cycles (compute)	24 cycles (compute)
Handshake Cycles	25 cycles (total)	25 cycles (total)
Throughput (24 cycles)	**6.67 Gbps**	**108.93 Gbps**
Throughput (25 cycles)	**6.40 Gbps**	**104.58 Gbps**

Table 4: Critical path gate-by-gate delay breakdown reported by OpenSTA.

Stage	**Instance**	**Cell Type**	**Fanout**	**Stage Delay (ns)**	**Cumulative Time (ns)**
1	_34916_/CK → Q	DFF_X1	1	0.0855	0.0855
2	_21498_	BUF_X2	5	0.0326	0.1182
3	_21499_	XNOR2_X2	1	0.0382	0.1564
4	_21500_	XNOR2_X1	1	0.0464	0.2028
5	_21503_	XNOR2_X2	2	0.0487	0.2515
6	_22765_	XOR2_X2	9	0.1047	0.3562
7	_31044_	XOR2_X2	4	0.0633	0.4195
8	_31045_	AND3_X1	1	0.0468	0.4663
9	_31047_	NOR3_X1	1	0.0102	0.4765
10	_31048_	AOI221_X1	1	0.0836	0.5601
11	_31049_	NOR2_X2	1	0.0103	0.5704
Total Data Arrival Time				**0.5704 ns**	
Library Setup Time (t_{setup})				**0.0416 ns**	
Minimum Clock Period ($T_{\min}$)				**0.6120 ns**	

4 Power and Energy Analysis

Cell-level static and dynamic power estimation was performed using OpenSTA at our target operating condition of 100 MHz ($T_{\text{clk}} = 10.0\,\text{ns}$) with a supply voltage of 1.1 V.

4.1 Power Consumption Breakdown

Table 5 presents the detailed power metrics categorized by cell group (sequential vs. combinational) and power mechanism (internal, switching, and leakage).

Table 5: Power consumption breakdown reported by OpenSTA at 100 MHz (1.1 V, 25°C).

Component Group	**Internal (mW)**	**Switching (mW)**	**Leakage (mW)**	**Total (mW)**	**Fracti**
Sequential (Registers)	1.490	0.034	0.125	1.649	10.8
Combinational Logic	6.860	6.270	0.521	13.651	89.2
Clock Network	0.000	0.000	0.000	0.000	0.0
Total Power	**8.350**	**6.304**	**0.645**	**15.300**	**100.0**
Percentage	*54.6%*	*41.2%*	*4.2%*	*100.0%*	

4.2 Energy per Permutation and Energy Efficiency

For a single 1600-bit state permutation at 100 MHz ($T_{\text{clk}} = 10.0\,\text{ns}$):

- **24 Compute Cycles ($T_{\text{exec}} = 240\,\text{ns}$):**

$$E_{\text{perm}} = P_{\text{total}} \times T_{\text{exec}} = 15.30\,\text{mW} \times 240\,\text{ns} = \mathbf{3.672}\,\text{nJ / permutation} \tag{12}$$

- **Energy per bit (1600-bit state):**

$$E_{\text{bit}} = \frac{3.672\,\text{nJ}}{1600\,\text{bits}} = \mathbf{2.295}\,\text{pJ / bit} \tag{13}$$

- **25 Total Cycles including Handshake ($T_{\text{total}} = 250\,\text{ns}$):**

$$E_{\text{total}} = 15.30\,\text{mW} \times 250\,\text{ns} = \mathbf{3.825}\,\text{nJ / permutation} \quad (2.391\,\text{pJ / bit}) \tag{14}$$

4.3 Static Leakage vs. Dynamic Power Analysis

1. **Low Static Leakage:** The total static leakage power is only 0.645 **mW** (constituting just 4.2% of total power). This low standby consumption makes the iterative round-per-cycle architecture well suited for battery-powered edge computing and secure cryptographic smart cards.

2. **Dynamic Switching Power:** Combinational dynamic power accounts for 89.2% of the total dissipation due to wide bit-parallel switching across the 1600-bit XOR trees. The absence of pipeline registers between round steps eliminates intermediate flip-flop clocking energy.

5 Functional Verification

The functionality of the `keccak_f1600` module was rigorously verified against the official testbench `tb_keccak_f1600.v` using **Icarus Verilog v12.0**. The testbench reads 10 standard 1600-bit test vectors from `in.txt`, drives the word-addressed write handshake across 25 words (50 clock cycles), pulses `start`, observes the single-cycle `ready` strobe, and samples words 0 through 24 back into `captured_out` to verify against `out.txt`.

Listing 3: Automated verification transcript from Icarus Verilog.

```
==========================================================
 Starting Keccak-f[1600] automated verification
==========================================================
Test 1: PASS
Test 2: PASS
Test 3: PASS
Test 4: PASS
Test 5: PASS
Test 6: PASS
Test 7: PASS
Test 8: PASS
Test 9: PASS
Test 10: PASS
==========================================================
 Keccak-f[1600] Verification Summary
==========================================================
 Total test cases   : 10
 Passed             : 10
 Failed             : 0
 Result             : ALL TESTS PASSED
==========================================================
```

All 10 test vectors passed with zero mismatches, verifying exact cycle-accurate compliance with NIST FIPS 202 and the assignment specifications.

6 Summary of Implementation Metrics

Table 6 consolidates the final implementation metrics for the Keccak-f[1600] core.

Table 6: Consolidated implementation metrics for the Keccak-f[1600] ASIC core.

Design Metric	**Value / Result**
Standard Cell Library	Nangate Open Cell Library (45 nm, Typical Corner)
Operating Conditions	1.1 V, 25°C
Design Architecture	Iterative Round-per-Cycle (24 computation rounds)
Total Standard Cells	**18,076 cells**
Total Silicon Area	**29,221.96 μm^2**
Gate Equivalents (GE)	**36,619 GE**
Sequential Area	7,303.03 μm^2 (1,615 DFF_X1 cells, 24.99%)
Combinational Area	21,918.93 μm^2 (75.01%)
Target Clock Period (T_{clk})	10.00 ns (100.0 MHz)
Worst-Case Data Arrival Time	0.5704 ns
Library Setup Time (t_{setup})	0.0416 ns
Worst-Case Timing Slack (@ 100 MHz)	**+9.3880 ns (MET)**
Minimum Clock Period (T_{min})	**0.6120 ns**
Maximum Operating Frequency (F_{max})	**1.634 GHz**
Throughput @ 100 MHz (24 cycles)	**6.67 Gbps**
Throughput @ 100 MHz (25 cycles)	**6.40 Gbps**
Peak Throughput @ F_{max} (24 cycles)	**108.93 Gbps**
Total Power Dissipation (@ 100 MHz)	**15.30 mW**
Static Leakage Power (@ 100 MHz)	0.645 mW (4.2%)
Dynamic Power (@ 100 MHz)	14.65 mW (95.8%)
Energy per Permutation (24 cycles @ 100 MHz)	**3.672 nJ / permutation**
Energy per Bit (1600 bits)	**2.295 pJ / bit**
Functional Verification Status	**10 / 10 PASS (100% match)**

7 Submission File Listing

The following deliverables are generated and verified in the project directory:

- `Report.pdf` – Comprehensive compiled PDF report.
- `keccak_f1600.v` – Synthesizable Verilog RTL implementation of the Keccak-f[1600] permutation core conforming to the exact module interface.
- `tb_keccak_f1600.v` – Official assignment testbench (unmodified except for file paths).
- `in.txt`, `out.txt` – 1600-bit golden test vectors.
- `keccak_f1600.sdc` – SDC timing constraints file (100 MHz clock).
- `abc.constr` – ABC timing and drive constraints file.
- `synth.ys` – Yosys synthesis script.
- `synth_keccak.v` – Synthesized gate-level netlist mapped to Nangate 45 nm standard cells.
- `synth.log` – Full Yosys synthesis log and cell statistics.
- `sta.tcl` – OpenSTA execution script.
- `sta.log` – Full OpenSTA timing and power analysis report.
- `NangateOpenCellLibrary_typical.lib` – 45 nm standard cell library file.

References

[1] National Institute of Standards and Technology (NIST), "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," *Federal Information Processing Standards Publication (FIPS 202)*, Aug. 2015. `https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf`

[2] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "The Keccak Reference," *Submission to NIST SHA-3 Competition*, Jan. 2011. `https://keccak.team/files/Keccak-reference-3.0.pdf`

[3] U. Banerjee, A. Wright, C. Juvekar, M. Waller, Arvind, and A. P. Chandrakasan, "An Energy-Efficient Reconfigurable DTLS Cryptographic Engine for Securing Internet-of-Things Applications," *IEEE Journal of Solid-State Circuits (JSSC)*, vol. 54, no. 8, pp. 2339–2352, Aug. 2019.

[4] B. Baldwin et al., "FPGA Implementations of the Round 2 SHA-3 Candidates," *Proc. NIST Second SHA-3 Candidate Conference*, 2010.

[5] Nangate Inc., "Nangate 45nm Open Cell Library Documentation," *Nangate Design Kit*, 2008.

[6] C. Wolf, "Yosys Open SYnthesis Suite," `https://yosyshq.net/yosys/`

[7] Parallax Software, "OpenSTA: Parallax Static Timing Analyzer," `https://github.com/The-OpenROAD-Project/OpenSTA`